

Combine MATLAB program for multivariate Calculus Lab

```
% Revision of partial derivatives for finding partial derivatives
syms x y z
f=input('Enter function of x, y, z')
%find first order partial derivatives
fx=diff(f, x)
fy=diff(f, y)
fz=diff(f, z)

%find higher order partial derivatives
n=input('entre order of partial derivative')
fx_n=diff(f, x, n)
fy_n=diff(f, y, n)
fz_n=diff(f, z, n)
```

```
%MATLAB Program-1
% Take no. of variables required to define function
n=input("Enter required no. of variables to define function")
var=sym('x',[1,n])
% define function
f=input("Enter your function in terms of x1, x2, ...,xn",'s');
f=str2sym(f)
% required partial derivative
o=input("Enter order of partial derivative");
for i=1:o
    m=input("Enter the variable no. w.r.t. which derivative to be taken
from right to left");
    d=diff(f,var(m));
    f=d;
end
fprintf("Required partial derivative of f is %s",d)
```

```
%MATLAB Program-1 (Alternate Option)
% Interactive Higher-Order and Mixed Partial Derivative Finder
syms x y z; % Define symbolic variables

% Prompt user to enter the function
f = input('Enter the function in terms of variables x, y, z (e.g., x^2 + y*z): ');
% Prompt user to specify the type of derivative
disp('Choose the type of derivative:');
disp('1: Higher-order derivative with respect to a single variable');
disp('2: Mixed partial derivative')
choice = input('Enter your choice (1 or 2): ')

if choice == 1
    % Higher-order derivative
    var = input('Enter the variable for differentiation (x, y, or z): ',
's');
```

```

    order = input('Enter the order of the derivative (e.g., 1, 2, 3, ...):
');
    higher_order_derivative = diff(f, var, order); % Compute higher-order
derivative
    disp(['The ', num2str(order), '-order partial derivative with respect to
', var, ' is:'])
    disp(char(higher_order_derivative))

elseif choice == 2
    % Mixed partial derivative
    vars_input = input('Enter the sequence of variables for differentiation
(e.g., "xxy" for d^3/dx^2dy): ', 's');
    mixed_derivative = f; % Start with the original function
    for i = 1:length(vars_input)
        mixed_derivative = diff(mixed_derivative, vars_input(i)) %
Differentiate successively
    end
    disp(['The mixed partial derivative with respect to ', vars_input, '
is:'])
    disp(char(mixed_derivative))

else
    disp('Invalid choice! Please restart the program and choose either 1 or
2.')
end

```

```

%MATLAB Program-2
% Check Second-Order Mixed Partial Derivatives for a Two-Variable Function

syms x y; % Define symbolic variables

disp('This program checks if second-order mixed partial derivatives are
equal for a function of two variables.')

% Prompt user to enter the function
func_input = input('Enter the function in terms of variables x and y (e.g.,
x^2 + y^3): ', 's')
f = str2sym(func_input); % Convert the input string to a symbolic expression

% Display the function
disp(['Your function is: ', char(f)])

% Compute the two mixed partial derivatives
mixed_xy = diff(diff(f, x), y) %  $\partial^2 f / \partial x \partial y$ 
mixed_yx = diff(diff(f, y), x) %  $\partial^2 f / \partial y \partial x$ 

% Display the computed derivatives
disp(['The partial derivative  $\partial^2 f / \partial x \partial y$  is: ', char(mixed_xy)])
disp(['The partial derivative  $\partial^2 f / \partial y \partial x$  is: ', char(mixed_yx)])

```

```
% Check if the mixed partial derivatives are equal
if isequal(mixed_xy, mixed_yx)
    disp('The second-order mixed partial derivatives are EQUAL.')
else
    disp('The second-order mixed partial derivatives are NOT EQUAL.')
end
```

```
% MATLAB program-3
% Finding extreme values of the
function syms x y
f = input('Enter the function f(x, y): ', 's');
f = str2sym(f) % Convert input to symbolic expression
```

$$f = y^3 + x^3$$

```
% First derivatives
fx = diff(f, x)
```

$$fx = 3x^2$$

```
fy = diff(f, y)
```

$$fy = 3y^2$$

```
% Solve for critical points
critical_points = solve([fx == 0, fy == 0], [x, y]);
disp('Critical points:');
```

Critical points:

```
disp(critical_points);
```

```
x: 0
y: 0
```

```
% Second derivatives fxx, fyy, fxy
r = diff(fx, x)
```

$$r = 6x$$

```
t = diff(fy, y)
```

$$t = 6y$$

```
s = diff(fx, y)
```

$$s = 0$$

```
% Hessian determinant
D = r * t - s^2
```

$$D = 36xy$$

```
% Evaluate at critical points
disp('Classification of critical points:')
```

Classification of critical points:

```
for i = 1:length(critical_points.x)
x0 = critical_points.x(i);
y0 = critical_points.y(i);
% Evaluate second derivatives and Hessian determinant
D_t = subs(D, [x, y], [x0, y0])
r_t = subs(r, [x, y], [x0, y0])
fprintf('At critical point (%i, %i):\n', x0,
y0); if D_t > 0
if r_t > 0
fprintf('Given function f(x,y) is minimum and fmin=%.2f',subs(f,[x, y],
[x0, y0]))
elseif r_t < 0
disp('Given function f(x,y) is maximum and fmax=%.2f',subs(f, [x, y], [x0,
y0]))
end
elseif D_t < 0
disp('Saddle
point') else
```

```
D_t = 0
```

```
r_t = 0
```

At critical point (0, 0):

No conclusion, further investigation required

```

%Matlab Program-4
% Input number of variables
n = input('Enter required number of variables to define function: ')
% Dynamically create symbolic variables
var=sym('x',[1,n])
% Input number of functions
k = input('Enter the number of functions: ')
% Define symbolic function vector
g = sym(zeros(k, 1))
for i = 1:k
f = input(['Enter function ', num2str(i), ' in x1, x2, ..., xn form: '], 's')
g(i) = str2sym(f) % Convert string input to symbolic expression
end
% Compute Jacobian matrix
J = jacobian(g, var)
% Display the result
disp(['The Jacobian matrix is: ', char(J)])

```

```

%Matlab Program-4(Alternate option)
% Input number of variables
syms x y
f1 = input('Enter the input function1 : ')

```

$$f1 = y^2 + x^2$$

```
f2 = input('Enter the input function2 : ')

```

$$f2 = x y$$

```
J=jacobian([f1, f2], [x, y])

```

```
J =
```

$$\begin{pmatrix} 2x & 2y \\ y & x \end{pmatrix}$$

```
Jaco = simplify(det(J))

```

$$Jaco = -2y^2 + 2x^2$$

```

% %Matlab Program-4(Alternate option)
% Take no. of Independent variables required to define function
n = input("Enter the no. of independent variables to define function: ");
Ivars = sym('x',[1, n]) % Create symbolic variables

```

$$Ivars = (x_1 \ x_2)$$

```

% Define function
funcs = [];
for i = 1:n
    f = input(sprintf("Enter the %d function in terms of x1, x2, ..., xn: ",
i), 's');
    f = str2sym(f); % Convert string input to symbolic expression
    funcs = [funcs,f]
end

```

```
funcs =  $x_1 x_2$ 
```

```
funcs =  $(x_1 x_2 \quad x_2 + x_1)$ 
```

```

J = jacobian(funcs, Ivars);
Jaco = simplify(det(J));
fprintf("The jacobian of given fuctions is %s",Jaco);

```

```
The jacobian of given fuctions is x2 - x1
```

```

% %Matlab Program-5 : Fourier Series

syms x n integer
% Program to find Fourier Series as per interval
% Input Interval
Ia = input("Enter starting value of the Interval: ");
Ib = input("Enter end value of the Interval: ");
fprintf("Given interval is (%s,%s) with period ", sym(Ia), sym(Ib));
L = (Ib - Ia) / 2; % Find Period

% Take function from user
disp("The function to find the Fourier series is");
f = input("Enter the function: ", 's');
f = str2sym(f);

% Check if interval is symmetric (-L, L)
if Ia == -Ib
    % Check if function is even or odd
    if subs(f, x, -x) == f
        disp("Function f(x) is an Even function, since f(-x) = f(x)");
    elseif subs(f, x, -x) == -f
        disp("Function f(x) is an Odd function, since f(-x) = -f(x)");
    else
        disp("Function f(x) is neither even nor odd");
    end
else
    disp("The interval is not symmetric, function parity check is skipped.");
end

% Find Fourier coefficients a0, an, and bn
a0 = (1 / L) * int(f, x, Ia, Ib);
a0 = simplify(a0)
an = (1 / L) * int(f * cos(pi * n * x / L), x, Ia, Ib)
an = simplify(expand(an));
bn = (1 / L) * int(f * sin(pi * n * x / L), x, Ia, Ib)
bn = simplify(bn);

% Display the Fourier series
disp("Required Fourier series of f(x) is:")
FS=a0/2 + symsum(an*cos((n*pi*x)/L)+bn*sin((n*pi*x)/L),n)

```

```
% %Matlab Program-6 : Harmonic Series
% Given data
x = [0, pi/3, 2*pi/3, pi, 4*pi/3, 5*pi/3]; % Discrete values of x
y = [1.0, 1.4, 1.9, 1.7, 1.5, 1.2];      % Corresponding values of y

N = length(x); % Number of data points
L = pi;        % Since x is from 0 to 2*pi => 2L = 2*pi => L = pi

% Compute a0
a0 = (2/N) * sum(y);

% Compute a1, a2, b1, b2
a1 = (2/N) * sum(y .* cos(x));
a2 = (2/N) * sum(y .* cos(2*x));
b1 = (2/N) * sum(y .* sin(x));
b2 = (2/N) * sum(y .* sin(2*x));
```

```
a0 = 2.9000
```

```
fprintf('a1 = %.4f\n', a1);
```

```
a1 = -0.3667
```

```
fprintf('a2 = %.4f\n', a2);
```

```
a2 = -0.1000
```

```
fprintf('b1 = %.4f\n', b1);
```

```
b1 = 0.1732
```

```
fprintf('b2 = %.4f\n', b2);
```

```
b2 = -0.0577
```

```
% Display Fourier Series expression
fprintf('\nFourier Series (up to 2nd harmonic):\n');
```

```
Fourier Series (up to 2nd harmonic):
```

```
fprintf('y(x) = %.4f + (%.4f)*cos(x) + (%.4f)*sin(x) + (%.4f)*cos(2x) +  
(%.4f)*sin(2x)\n', ...  
        a0/2, a1, b1, a2, b2);
```

```
y(x) = 1.4500 + (-0.3667)*cos(x) + (0.1732)*sin(x) + (-0.1000)*cos(2x) + (-0.0577)*sin(2x)
```



```
% %Matlab Program-6 : Beta & Gamma
% Please note, Gamma and Beta are one kind of integrations
% so, use regular format of solving integration
%Beta Function
% declare the variable
syms x

% take function from user
f=input("Enter your function f(x)", 's');
```

$$f = \sqrt{x} e^{-\sqrt{x}}$$

```
% take limits of integration
a=input("Enter lower limit")
```

```
a =
0
```

```
b=input("Enter upper limit")
```

```
b =
Inf
```

```
% Evaluate integration
I=int(f,x,a,b)
```

```
I = 4
```

```
fprintf("Integral from %s to %s of (%s) is %s", sym(a), sym(b), f, I)
```

```
Integral from 0 to Inf of (x^(1/2)*exp(-x^(1/2))) is 4
```

```
syms t x
% Gamma Function
I1=int(exp(-t)*t^4,t,0,Inf)
```

```
I1 = 24
```

```
% or
I2=gamma(5)
```

```
I2 =
24
```

```
% Beta Function
I3=double(int(x^3*(1-x)^4,x,0,1))
```

```
I3 =
0.0035714
```

```
% Or
I4=beta(4,5)
```

```
I4 =
    0.0035714
```

```
% MATLAB Program-7 DUIS Rule
```

```
syms x c
syms a b positive      % Positive parameters
disp("Given function/Integrand is :")
```

Given function/Integrand is :

```
Ig=input("Enter the function/Integrand f(x)", 's');
Ig=str2sym(Ig)      % Ig means given I
```

$$I_g = e^{-b x^2} \cos(2 a x)$$

```
%Take limits from user
Ia=input("Enter the value of lower limit")
```

```
Ia =
0
```

```
Ib=input("Enter the value of upper limit")
```

```
Ib =
Inf
```

```
p=input("Differentiation w.r.t. which parameter/variable?");
fprintf("differentiating w.r.t. %s, using DUIS rule",p)
```

differentiating w.r.t. a, using DUIS rule

```
if has(sym(Ia),p) || has(sym(Ib),p)
    dI_dp=int(diff(Ig,p),x,Ia,Ib)+diff(Ib,p)*subs(Ig,x,Ib)-
    diff(Ia,p)*subs(Ig,x,Ia);
    fprintf("df_d%s = %s",p,dI_dp)
else
    dI_dp=int(diff(Ig,p),x,Ia,Ib);
    fprintf("df_d%s = %s",p,dI_dp)
end
```

$$df_{da} = -(a \pi^{1/2} \exp(-a^2/b)) / b^{3/2}$$

```
Ic=int(dI_dp,p) + c      % Ic means I calculated
```

```
Ic =
```

$$c + \frac{\sqrt{\pi} e^{-\frac{a^2}{b}}}{2 \sqrt{b}}$$

Alternative option for DUIS

```
% To calculate c
q=input(['Enter the appropriate value of ', char(p), ' to evaluate c']);
fprintf("Put %s=%s, to calculate c",char(p),sym(q))
```

Put a=0, to calculate c

```
disp("Ig=Ic")
```

Ig=Ic

```
eq=int(subs(Ig,p,q),x,Ia,Ib)==subs(Ic,p,q)
```

eq =

$$\frac{\sqrt{\pi}}{2\sqrt{b}} = c + \frac{\sqrt{\pi}}{2\sqrt{b}}$$

```
c1=solve(eq,c)
```

c1 = 0

```
disp("Required integral is ")
```

Required integral is

```
I=subs(Ic,c,c1);
I=simplify(I)
```

I =

$$\frac{\sqrt{\pi} e^{-\frac{a^2}{b}}}{2\sqrt{b}}$$

% MATLAB Program number-8, Double integration

```
syms x y % Note that for polar form put r=x and theta=y
f=input("Enter the function",'s');
f=str2sym(f);
```

```
f = x y
```

$$I = \int_{y_l}^{y_u} \int_{x_l}^{x_u} f \, dx dy$$

```
x1=input("Enter the Lower limit of x")
```

```
x1 =
0
```

```
xu=input("Enter the Upper limit of x")
```

```
xu =
1
```

```
y1=input("Enter the Lower limit of y")
```

```
y1 =
0
```

```
yu=input("Enter the Upper limit of y")
```

```
yu = 1 - x
```

```
if isnumeric(x1) && isnumeric(xu) && isnumeric(y1) && isnumeric(yu)
    int(int(f,x,x1,xu),y,y1,yu)
elseif isnumeric(y1) && isnumeric(yu)
    int(int(f,x,x1,xu),y,y1,yu)
elseif isnumeric(x1) && isnumeric(xu)
    int(int(f,y,y1,yu),x,x1,xu)
else
    disp("Please check the limits!")
end
```

```
ans =
```

```
 $\frac{1}{24}$ 
```

Note: For finding area, take function **f=1** in above program and refer the given diagram for finding limits.

% MATLAB Program number-9, Triple integration (For Practice purpose only)

```
syms x y z a % Note that for polar form replace r=x, theta=y and phi = z
f=input("Enter the function / Integrand",'s');
f=str2sym(f);
f=simplify(f)
```

$$f = x^2 y z$$

$$I = \int_{x_l}^{x_u} \int_{y_l}^{y_u} \int_{z_l}^{z_u} f \, dx \, dy \, dz$$

```
x1=input("Enter the Lower limit of x")
```

```
x1 =
0
```

```
xu=input("Enter the Upper limit of x")
```

```
xu =
1
```

```
y1=input("Enter the Lower limit of y")
```

```
y1 =
0
```

```
yu=input("Enter the Upper limit of y")
```

$$y_u = 1 - x$$

```
z1=input("Enter the Lower limit of z")
```

```
z1 =
0
```

```
zu=input("Enter the Upper limit of z")
```

$$z_u = 1 - y - x$$

```
if isnumeric(x1) && isnumeric(xu) && isnumeric(y1) && isnumeric(yu) &&
isnumeric(z1) && isnumeric(zu)
```

```
I=int(int(int(f,x,sym(x1),sym(xu)),y,sym(y1),sym(yu)),z,sym(z1),sym(zu)) %
sym() used for pi kind of limits
```

```
elseif isnumeric(x1) && isnumeric(xu)
    if has(or(z1,zu),x) || has(or(z1,zu),z) && has(or(y1,yu),x)
```

```
I=int(int(int(f,z,sym(z1),sym(zu)),y,sym(y1),sym(yu)),x,sym(x1),sym(xu))
    elseif has(or(y1,yu),x) || has(or(y1,yu),z) && has(or(z1,zu),x)
```

```
I=int(int(int(f,y,sym(y1),sym(yu)),z,sym(z1),sym(zu)),x,sym(x1),sym(xu))
end
```

```

elseif isnumeric(yl) && isnumeric(yu)
    if has(or(zl,zu),x) || has(or(zl,zu),y) && has(or(xl,xu),y)

I=int(int(int(f,z,sym(zl),sym(zu)),x,sym(xl),sym(xu)),y,sym(yl),sym(yu))
    elseif has(or(xl,xu),y) || has(or(xl,xu),z) && has(or(zl,zu),y)

I=int(int(int(f,x,sym(xl),sym(xu)),z,sym(zl),sym(zu)),y,sym(yl),sym(yu))
    end
elseif isnumeric(zl) && isnumeric(zu)
    if has(or(yl,yu),x) || has(or(yl,yu),z) && has(or(xl,xu),z)

I=int(int(int(f,y,sym(yl),sym(yu)),x,sym(xl),sym(xu)),z,sym(zl),sym(zu))
    elseif has(or(xl,xu),y) || has(or(xl,xu),z) && has(or(yl,yu),x)

I=int(int(int(f,x,sym(xl),sym(xu)),y,sym(yl),sym(yu)),z,sym(zl),sym(zu))
    end
else
    disp("Please check the limits!")
end

```

I =

$$\frac{1}{2520}$$